

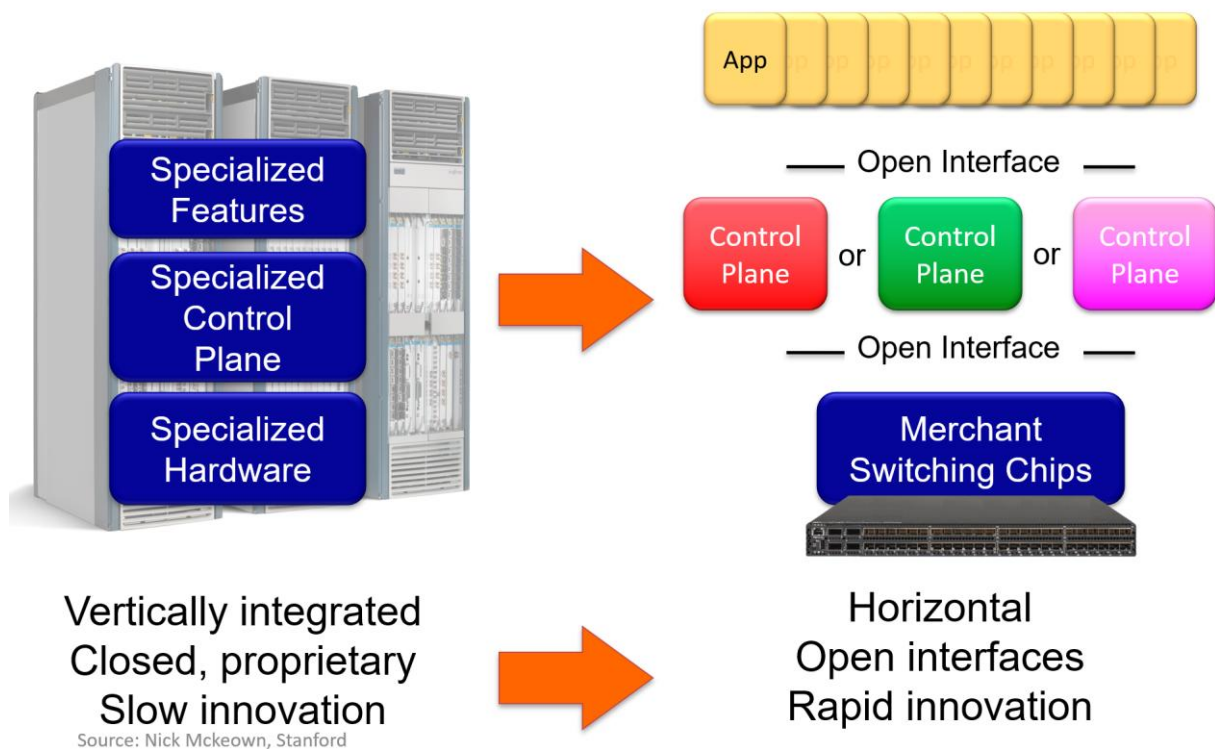
Outline:

- Goal: Open vSwitch usage, command line options, and additional utilities,

Study environment:

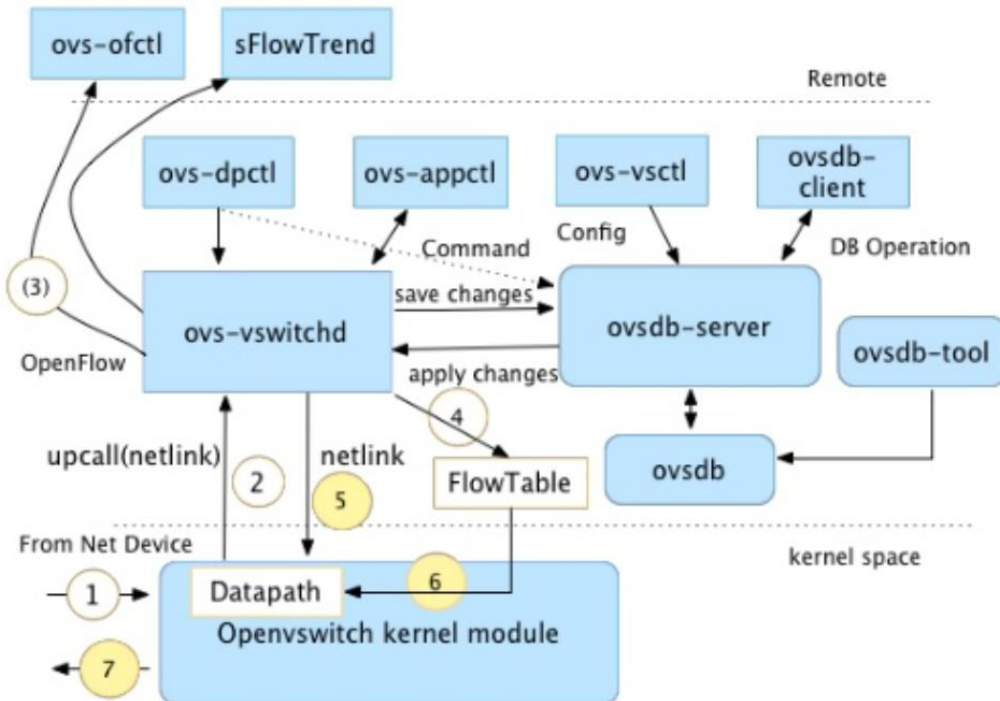
- all materials are uploaded on UPEL Moodle;
- use Ubuntu image on desktops (*inf3/inf3*)
- At the end of labs upload the text document with commands to the UPEL
- tip: before exercises, clean Mininet environment using `sudo mn --clean`
- tip: copy all your commands to a texteditor, the terminal is sometimes not reliable

Intro to Software Defined Networking (SDN)



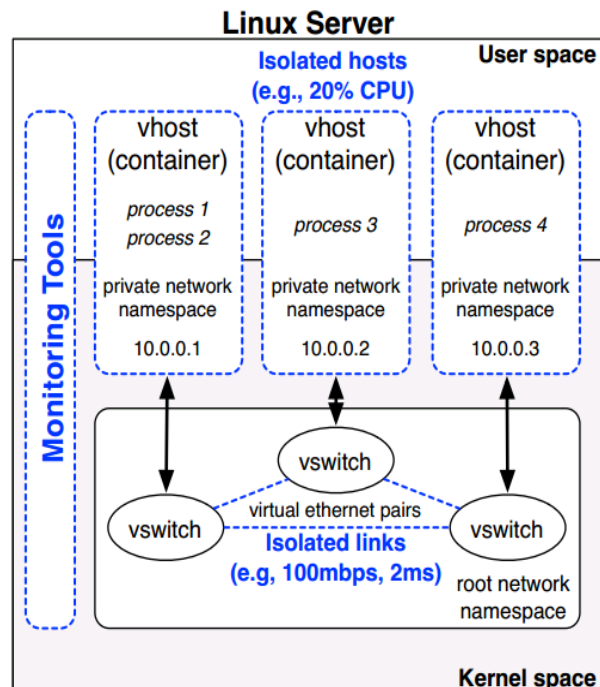
Get familiar with the Open vSwitch environment

Open vSwitch is the most popular software switch. It is by default included in many Linux distributions. It is a core component of open virtualization environments (such as KVM, XEN ...). On Linux operating systems, it utilizes both user and kernel space to deliver optimal packet-switching performance in virtualized environments. It has a very powerful configuration interface based on command-line tools.



Picture 1 OvS Architecture

Another utility which we will use is Mininet. It allows the creation of machine-local virtual networks with arbitrary topologies. It is a great tool for testing and development of SDN-based applications. It is based on light virtualization and allows easy access to mid-size topologies.



Task 1 Basic Mininet interface

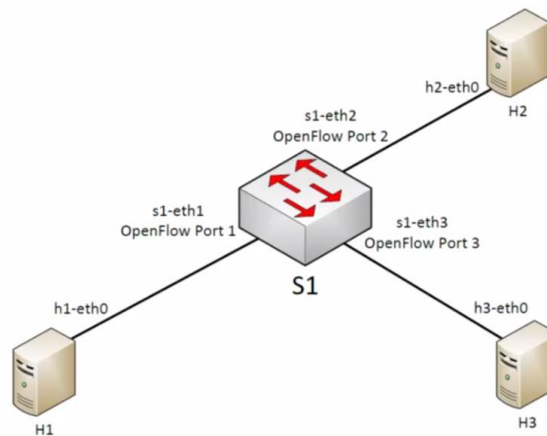
To the most important commands, we can include *ovs-vsctl*. In general it configures *ovs-vswitchd*, but really it is a high-level interface for database. It has a huge set of options and parameters which allows *ovs-vsctl* to monitor and manage an OVSDB database.

Using *ovs-vsctl* we can easily add, modify and monitor different elements of the Linux networking stack.

Next launch mininet, and execute following command:

```
sudo mn --topo=single,3 --controller=none --mac
```

It will create following topology



Picture 2 Mininet topology

Once you execute the command you will have access to the command line inside the environment.

You can display the virtual environment configuration using *dump* and *net* command:

```
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=19960>
<Host h2: h2-eth0:10.0.0.2 pid=19962>
<Host h3: h3-eth0:10.0.0.3 pid=19964>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None
pid=19969>

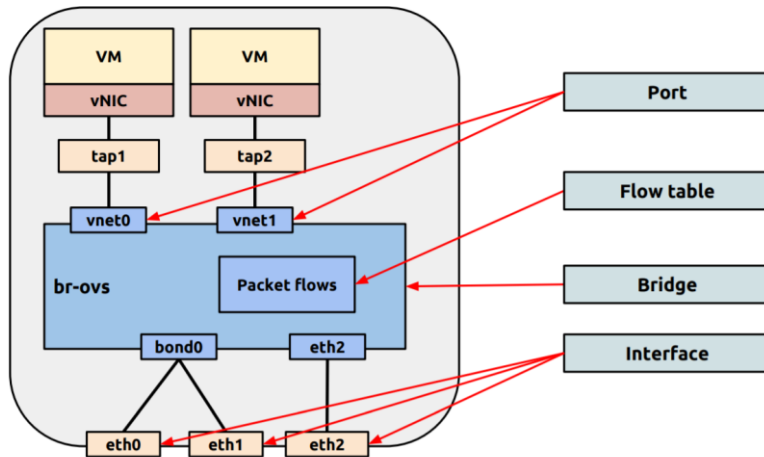
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
```

If you want to execute some command in the context of host or switch add name as a first argument.

List interfaces on *h1* (*h1 ifconfig*).

- As you may noticed all hosts have linear MAC address it is due to the option `--mac`
- Invoke *pingall* command and test connectivity between all hosts. Is there connectivity between the hosts? There should be no connectivity as there is no control plane yet (`--controller=none`)

TASK 1.1 - list interfaces on the h2 host



By adding `sh` prefix you can execute commands in the hosting shell. Using `ovs-vsctl` list all bridges and ports on the switch. Now list all bridges in the environment:

```
sh ovs-vsctl show
```

- How many ports there are on your bridge?

Another `ovsdb-client` can be used to display information regarding the OVSDb database scheme.

You can display entire schema using `ovsdb-client get-schema --pretty` (do not execute it yet). Here is short descriptions of the most important fields:

- *Open_vSwitch* - Open vSwitch configuration
- *Bridge* - Bridge configuration
- *Port* - Port configuration
- *Interface* - One physical network device in a Port
- *QoS* - Quality of Service configuration
- *Queue* - QoS output queue
- *Mirror* - Port mirroring
- *Controller* - OpenFlow controller configuration
- *Manager* - OVSDb management connection
- *NetFlow* - NetFlow configuration
- *SSL* - SSL configuration
- *sFlow* - sFlow configuration

Now display all ports using

```
sh ovsdb-client dump Port name interfaces tag vlan_mode bond_mode
```

OVSDb database is log-based and allows tracking the changes. Use the following command to identify the latest changes made in database:

- `sh sudo ovsdb-tool show-log -m`

As you can see, you track changes in the database overtime.

TASK 1.2 - list all bridges configuration in you system (use ovs-vsctl command)

Task 2 Forwarding architecture

Command *ovs-ofctl* speaks to OpenFlow module and provides a simple interface for adding flows. Here is the list of the most common options (do not enter them yet; you will need them later):

- *ovs-ofctl show <bridge>*
- *ovs-ofctl dump-flows <bridge>*
- *ovs-ofctl add-flow <bridge> <flow>*
- *ovs-ofctl del-flows <bridge> [flow]*

Open vSwitch can work on two modes:

- *NORMAL* – all traffic is forwarded using standard way – as a learning switch
- *FLOW* - specific rules are applied on flow basis.

Currently the switch is running in the FLOW mode. Now try to execute following command:

```
sh ovs-appctl fdb/show s1
```

The command will display the current entries in the forwarding table (fdb) of the switch (not flow table), it contains regular L2 switch entries with information about the MAC address and ports

- How many entries you can see?

Now try to install following flow using *ovs-ofctl*

```
sh ovs-ofctl add-flow s1 action=normal
```

It defines a flow that matches all traffic and applies NORMAL action. Inside Mininet execute *pingall* command. Now, try to ping *h2* from *h1*

- Can you ping hosts?
- How many flow entries you can see (*sh ovs-ofctl dump-flows s1*)?
- How many packets were transmitted?
- What are the changes in the forwarding table of *s1* ?

Now try to delete flows using following command:

```
sh ovs-ofctl del-flows s1
```

- How many flows you can see now?
- Can you ping hosts?

Now let's start using Ovs in the FLOW mode, and install some flows. The flow contains two mandatory section *MATCH* and *ACTION*. Once there is matching action is executed. You can match on different fields of packets, in action you can specify the ie. physical port to send packets to, actions to be executed on packets such as *drop* or other table in processing (from OpenFlow 1.1). In

addition, each flow has certain properties, such as *priority*. The priority allows to control when the rule will be applied. Now try to add the following flow with priority 500:

```
sh ovs-ofctl add-flow s1 priority=500,in_port=1,actions=output:2
```

The flow will send all packet from port 1 (*in_port=1*) to port 2 (*actions=output:2*), we are performing matching strictly on *L1* here

In order to establish connectivity you have to add returning flow.

- Check the connectivity between h1 and h2? You should be able to ping hosts
- How many flow entries can you see?

As mentioned earlier kernel datapath processing is the essential feature of OVS. Now let's display the datapath in kernel settings invoke following command:

```
sh ovs-dpctl show
```

and compare it with switch port settings:

```
sh ovs-ofctl show s1
```

As you may see in kernel space, all interfaces are represented as a flat list, not grouped by bridge boundaries.

Try to answer the following questions:

- What you can say about the lookups and cache performance?
-

Now let's compare flows from *vswitchd* and flows in the kernel path. Execute

```
sh sudo ovs-dpctl dump-flows
```

Notice the difference, kernel path are used for resending data in kernel space. NOTE: kernel path are deleted when not active, please send fresh traffic if the list is empty.

Now let's try to block the traffic by installing the flow with higher priority by executing command:

```
sh ovs-ofctl add-flow s1 priority=700,actions=drop
```

you will now install the flow that block communication

- Check the connectivity? You should not be able to ping hosts
- How many flow entries you can see?

Now try to delete blocking flow using *ovs-ofctl* using priority definition and option *--strict*

- How many flow entries you can see?

TASK 2 - list all flows in kernel datapath and in vswitchd

Task 3 Filtering based on L2

Restart the Mininet environment (from command line out of Mininet execute `sudo mn -c`). Using `ovs-ofctl` command add flows that will enable traffic between all hosts based only on L2 matching. In previous task you specified flow based on `in_port` property now try to use `dl_dst` and `dl_src` (dl stand for *data link*).

- Add flow that specifies `dl_dst` and forward packets to certain port.
- Display OvS flows. Are you able to ping all hosts (use `pingall`) ?

Because ARP tables on hosts are empty you are not able to ping. Add following entry to your switch:

```
sh ovs-ofctl add-flow s1 dl_type=0x806,actions=flood
```

It will flood all ARP messages

- Are you able to ping all hosts ? You should be able to ping all hosts.

TASK 3 - (1) list all flows in flow table

Task 4 L3 matching and packet modification

Restart the Mininet environment (from command line out of Mininet execute `sudo mn -c`). Define flow which allows the nodes in subnet 10.0.0.0/24 to talk to one another using IP protocol. You can use following command

```
sh ovs-ofctl add-flow s1  
dl_type=0x800,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2
```

Add all missing flows. Now verify connectivity

- Are you able to ping hosts?

As in the previous task, you cannot ping hosts because ARP flooding is not enabled. Add support for ARP entries without flooding the entire network by filtering to specific ports using the following approach:

- Each ARP request contains the IP address of the host we are asking, and forward ARP requests directly to ports using `nw_dst` matching, e.g., `nw_dst=10.0.0.1`, since you know all the hosts and ports with the type of data link set to ARP
- Display flows, you should be able to ping hosts

TASK 4 - (1) list all flows in flow table.

Task 5 L4 matching

Restart the Mininet environment (from the command line outside of Mininet, execute `sudo mn -c`). Start Python webserver on h3 by executing the command:

```
mininet> h3 python -m http.server 80 &
```

Similarly to the previous task, add *non-flooding ARP support*. Forward all HTTP traffic to port 3 using the following command:

```
sh ovs-ofctl add-flow s1
priority=500,dl_type=0x800,nw_proto=6,tp_dst=80,actions=output:3
```

Forward all IP traffic originated from h3 (*nw_src=10.0.0.3*) using mechanisms from Task 4

- Check service availability using *wget*. Hosts h1 and h2 should be able to connect to h3 on port 80.
- Allow only HTTP from h1/h2 to h3, block ICMP, keep ARP working, and prove it with *wget* and *ping*.

TASK 5 - (1) list all flows in flow table.